REDS
Institut
Reconfigurable
and Embedded
Digital Systems

Prof. Marina Zapater
Assist. Guillaume
Chacun, Mehdi
Akeddar

Artificial Intelligence for Autonomous Systems

Spring 2026

Lab 1 - Let's get started

Available time : **1 lab session**.

1 Lab objectives

In this lab, the objective is to familiarize oneself with the available hardware and development environment. You'll be working with DuckieBots (`https://duckietown.com/platform/`).

Disclaimers (applicable to all labs) :

1. During the semester, you will work in groups of two. **Working alone is not permitted.** As there is an odd number of students, there will be one group of 3. We expect the workload to be distributed fairly within the groups.
2. We reserve the right to evaluate students separately based on their work, or during oral evaluations, if we suspect an unfair distribution of the workload, a lack of individual contribution, or the abusive use of AI.
3. Any use of AI must be documented and credited. Failure to do so may result in penalties applied to the lab grade.
4. You will work with your own DuckieBot. These are fragile, so we ask you to always debug and test your code before deploying it to avoid unnecessary crashes.
5. We highly encourage you to charge the Duckiebot before coming to the lab session, as a 0 to 100% can take several hours.
6. The DuckieTown environment requires Ubuntu 22 to work on. We provide a ready-to-use VM. If you want to install natively on your laptop, please prepare it before the first lab session (`https://docs.duckietown.com/daffy/opmanual-duckiebot/setup/setup_laptop/index.html`).

2 Account

2.1 Github account

DT needs github account to interact with. Please create a github account if you don't have one already `https://github.com/`

2.2 DT account

We need a Duckietown Hub account to generate a personal token to connect to the Duckiebot. Please follow the guidelines at this link `https://docs.duckietown.com/ente/duckietown-manual/10-setup/01-accounts/duckietown-hub-account.html`

2.3 Docker account

You will also need a DockerHub account. Follow the steps here `https://docs.duckietown.com/ente/duckietown-manual/10-setup/01-accounts/docker-account.html` to get one.

3 Connect to Duckiebot

3.1 Network

Development on the Duckiebot depends on WIFI connection. Your laptop and the Duckiebot need to be on the same network. To connect the Duckiebot to a new WIFI, you'll need to :

- Insert the SD card in your laptop
- Navigate to /etc/wpa_supplicant.conf in the APP partition
- New networks can be created by adding a new network= paragraph, and then entering the network information. An example network configuration is shown in `https://docs.duckietown.com/daffy/opmanual-duckiebot/debugging_and_troubleshooting/network_config/index.html`

To verify if you're connected to the robot and ready to use it, you can use the following command :

```
dts fleet discover
```

You should be able to see your Duckiebot name displayed, with a green READY flag.

3.2 Dashboard

On a navigator, navigate to the following address :

```
http://[ROBOT_NAME].local/dashboard/
```

Take some time to discover the information presented here, especially the camera and the wheels modules. Make sure that you are able to see the output of the camera.

3.3 Demos

To make sure everything is setup and ready to go, we ask you to run a simple demo to control the robot with the keyboard. Open a terminal and run the following command :

```
dts duckiebot keyboard_control ![DUCKIEBOT_NAME]
```

Use arrows to move the robot around.

4 Build a DT project

We will be working with the DT template. Each lab will be provided as a git repo, forked from the DT template.

5 Your first ROS nodes

Now that we are sure everything is set up correctly, we can start developing our first application for the DuckieBot. The development is based on ROS. ROS, or Robot Operating System, is a flexible

framework for writing robot software. It is a collection of tools, libraries, and conventions that aim to simplify the creation of complex, robust robot behavior across various robotic platforms. ROS provides services designed for a heterogeneous computer cluster, such as hardware abstraction, low-level device control, implementation of commonly used functionality, message passing between processes, and package management. It is not an operating system in the traditional sense but rather a middleware that runs on top of an existing OS, typically Linux. Here's an example of a graph showing how ROS could work :

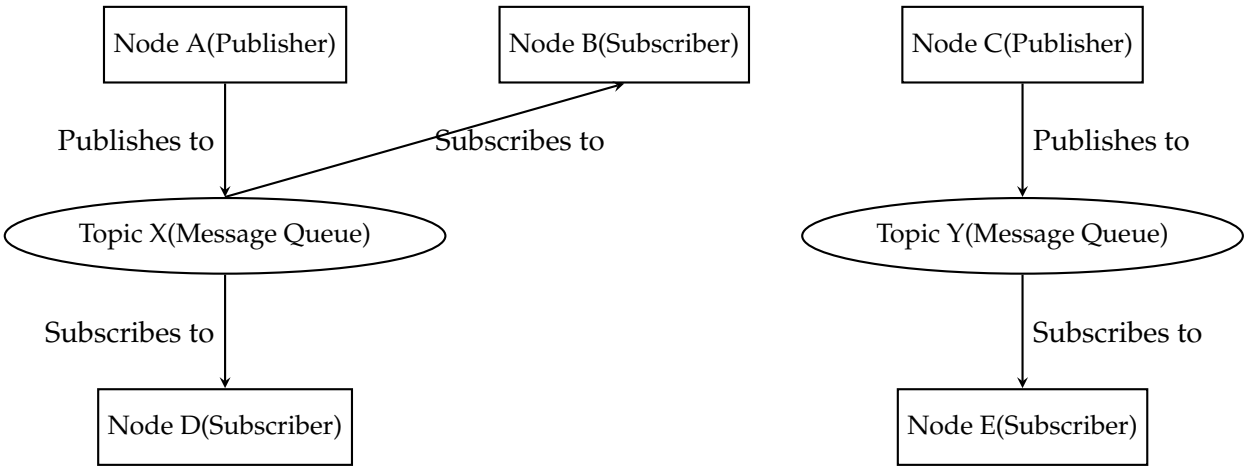

FIGURE 1 – ROS Node Communication Graph

— Nodes : Nodes are processes that perform computation in ROS. Each node is responsible for a specific task, such as controlling a sensor, processing sensor data, or controlling actuators.
— Topics : Topics are named buses over which nodes exchange messages. A node can publish messages to a topic and/or subscribe to a topic to receive messages.
— Messages : Messages are the data structures used to communicate between nodes. They are published to topics and can contain various types of data, such as sensor readings, control commands, or status information.

5.1 Publisher

Let's write our first Publisher. A publisher has the job of publishing messages from a ROS node into the ROS network for other nodes to receive.

Create the src directory inside my_package, and create the my_publisher_node.py. This file represents the behavior of your publisher.
Tasks : Implement the code for the publisher :

1. Create a ROS publisher that sends String messages
2. Define topic name as my_first_topic and queue size of 10.
3. Create a message that includes your robot's name. Use environment variables to make your code more flexible
4. Implement a publishing loop at 1 Hz. Use rospy.Rate for timing control
5. Add logging to track published messages

Make it executable :

```
chmod +x ./packages/my_package/src/my_publisher_node.py
```

Explore the launcher in ./launchers/my-publisher.sh

Now we can run our subscriber. Make sure the robot is reachable.

```
ping ROBOT_NAME.local
```

Compile your project

```
dts devel build -H ROBOT_NAME -f
```

Run it using the -l flag, to only run your launcher :

```
dts devel run -H ROBOT_NAME -L my-publisher
```

5.2 Subscriber

Let's write our first Subscriber. A subscriber has the job of listening for messages about a specific topic that are published by other ROS nodes (using ROS Publishers) over a ROS network. Again, nodes are placed inside the directory src/ of a Catkin package. Create the file my_subscriber_node.py and implement the subscriber code to listen to our publisher.
Tasks :

1. Create a ROS subscriber that listens for String messages
2. Connect it to the same topic as your publisher
3. Implement a basic callback function that log the heard data

Make it executable :

```
chmod +x ./packages/my_package/src/my_subscriber_node.py
```

Explore your launcher in ./launchers/my-subscriber.sh

Now we can run our subscriber. Make sure the robot is reachable.

```
ping ROBOT_NAME.local
```

Compile your project

```
dts devel build -H ROBOT_NAME -f
```

Run it using the -l flag

```
dts devel run -H ROBOT_NAME -L my-subscriber
```

6 Important information about build and run instructions

This section gives you some insight into what happens during build and run instructions :

— The -H flag indicates that your code is built to run on the robot itself, so you'll need to account for performance issues.

— You can run some of the nodes (or all of your code) on your computer to act as an external resource. Here, you can expect some latency issues, but it's quicker for development considerations. You can do this by removing the -H flag. The command become :
Compile your project

```
dts devel build -f
```

Run

```
dts devel run -L my-subscriber -R robot_name
```

— During this semester, we will alternate between the two methods, in order to get a sense of real-world deployment on embedded devices.

— The launchers provided for this lab are simple; each one launches one ROS node. We will explore how to define much more complex launchers, with dependencies on variables, etc., in the next labs. We encourage you to take some time to explore these concepts on the official documentation.

7 REMINDER - Calibration

Every camera is a little different and requires calibration, which we already did on all of your robots. If, for some reason, you had to do this procedure again, you can find a tutorial here `https://docs.duckietown.com/ente/duckietown-manual/20-operations/04-calibrations/duckiebot-ca` `html`. Every motor and encoder is different; you will need to perform the wheel calibration if you feel that your robot doesn't go straight when the input is 'go straight'. You can find a tutorial here `https://docs.duckietown.com/daffy/opmanual-duckiebot/operations/calibration_wheels/index.html`

8 Known issues

8.1 Blue light, fan spinning but robot won't boot up

The Jetson Nano may need a reflash. Please bring us the robot to do that for you, or follow the instructions in `https://docs.duckietown.com/ente/opmanual-duckiebot/06-troubleshooting/reflashing-jetson-nano-development-kit-4gb-duckietown.html`

8.2 Bad gateway error on the dashboard or noBT displayed on the screen

`https://docs.duckietown.com/daffy/opmanual-duckiebot/debugging_and_troubleshooti` `reset_dashboard.html`

```
docker -H DUCKIEBOT_NAME.local stop dashboard
docker -H DUCKIEBOT_NAME.local rm dashboard
docker -H DUCKIEBOT_NAME.local volume rm duckietown_compose-data
dts duckiebot update DUCKIEBOT_NAME
```

8.3 Force shutdown of a Duckiebot

Prefer using the dashboard to shutdown your Duckiebot. If for reason this does not work, SSH to the robot and shut it down using command line :

```
ssh duckie@DUCKIEBOT_NAME.local
    pwd: quackquack
sudo shutdown now
```

If you cannot access the robot via SSH, connect it to your network using an Ethernet cable. The steps are described here `https://docs.duckietown.com/daffy/opmanual-duckiebot/debugging_and_troubleshooting/network_config/index.html#option-2-bridge-the-internet-conne` Once you have connected via SSH, you can shut it down via the command line.